

IEOR E4576: Assignment 2

Table of Contents

Executive Summary	2
Methodology and Data Controls	3
Question 1 — CFTC Commitments of Traders as a Positioning Signal	4
1(a) Pull, Pivot, and Verify the Accounting Identities	4
1(b) Managed-Money Net Position, Front-Month Price, and COT Index	4
1(c) Commercial Hedging Pressure	6
1(d) Contemporaneous Returns and Managed-Money Flow	6
1(e) Predictive Regressions	8
Question 2 — Seasonal Calendar Spreads into Expiry	11
2(a) Natural Gas — NG[H][J0]	12
2(b) Corn — ZC[U][Z0]	13
2(c) Wheat — ZW[K][N0]	13
2(d) Lean Hogs — HE[V][Z0]	15
Data Integrity, Reproducibility, and Source Notes	16
Appendix A — Complete Python Code	17

Executive Summary

Replication repository: Full executable notebook, supporting code, and environment requirements are available on [GitHub](https://github.com/ajo2159/IEOR-E4576-HW2) (<https://github.com/ajo2159/IEOR-E4576-HW2>).

Bottom line. COT positioning is strongly contemporaneously related to WTI returns, but the predictive evidence is weak once overlapping-return inference is handled correctly. Seasonal calendar-spread effects are much stronger: the expected direction holds in 76%–96% of full-window annual instances across the four assigned spreads.

Table 1. Question 1 results at a glance

Question	Primary result	Interpretation
1(a)	Legacy and disaggregated OI identities: 100.0% within 1 contract	COT categories reconcile exactly to reported open interest.
1(b)	MM net / OI and full 3-year COT index constructed on an audited nearby CL series	Positioning varies materially over the cycle; the price overlay is descriptive, not causal.
1(c)	Commercials net short in 96.7% of weeks	Persistent commercial short hedging is a structural feature of crude positioning.
1(d)	$\rho = +0.3959$; slope = +1,497 contracts per +1% return	Price and managed-money positioning move together mainly within the same reporting week.
1(e)	Largest HAC t = 1.22 under assignment timing; 1.27 in post-release robustness	No strong robust predictive relationship at 1-, 4-, or 13-week horizons under either timing convention.

Table 2. Question 2 seasonal-spread results

Spread	Expected move	Agreement	Mean P&L	t-stat
NG[H][J0] Mar/Apr	Down	25/27 = 92.6%	−0.4183	−2.77
ZC[U][Z0] Sep/Dec	Down	25/26 = 96.2%	−11.9327	−4.83
ZW[K][N0] May/Jul	Down	21/26 = 80.8%	−12.8558	−2.45
HE[V][Z0] Oct/Dec	Up	19/25 = 76.0%	+2.3450	+2.53

P&L is measured in each futures spread's quoted price units over the common (−120, 0) trading-day window. The one-sample t-statistics are cross-year statistics on annual window P&Ls, not time-series t-statistics on the individual daily observations.

Methodology and Data Controls

Sample coverage. COT observations run from 2006-06-06 through 2026-09-08. Question 1 CL settlements are frozen at 2026-04-03; the completed-roll return history ends on 2026-03-13 because the final completed roll window must be fully observed. Predictive regressions therefore stop automatically rather than carrying stale prices into the COT tail.

Implementation controls used throughout

Control	Specification
Front-month schedule	Non-overlapping CL holding intervals; roll 5 observed CL sessions before LTD.
Return construction	Self-financing roll-consistent return index; old/new contract level gaps are excluded from outright returns.
COT index	156-week minimum history for a full trailing 3-year range.
Holiday alignment	Latest prior settlement allowed only within 3 calendar days.
Predictive horizons	Exact Friday + 7h calendar-day endpoints for $h \in \{1, 4, 13\}$.
Inference	OLS t-stat plus HAC/Newey-West robustness; overlap-motivated bandwidth $h=1$.
Information timing	Primary assignment convention starts from the following Friday; robustness starts from the first CL settlement strictly after that Friday.
Q2 full-window audit	Instructor-supplied futsread package is used unchanged; notebook requires 100% coverage of all 121 offsets from -120 through 0 .
Reproducibility snapshot	Q1 COT capped at 2026-09-08 and CL settlements at 2026-04-03; row counts, date spans, package versions, and SHA256 hashes are recorded.

Validation philosophy. The notebook is designed to fail closed on ambiguous data conditions: duplicate keys, unpriceable rolls, stale matches, non-monotone dates, or out-of-range COT-index values trigger explicit checks rather than silent repair.

Question 1 — CFTC Commitments of Traders as a Positioning Signal

Replication repository: Full executable notebook, supporting code, and environment requirements are available on [GitHub](https://github.com/ajo2159/IEOR-E4576-HW2) (https://github.com/ajo2159/IEOR-E4576-HW2).

1(a) Pull, Pivot, and Verify the Accounting Identities

I pull CL (contrcode 1986) CFTC data from June 2006 onward, retain futures-only observations, parse report type into category and side, and pivot to one row per COT snapshot. The final panel contains 1,057 report weeks and 21 futures-only report columns.

$$\text{Legacy: } OI = CM_L + NC_L + NC_X + NR_L = CM_S + NC_S + NC_X + NR_S$$

For the disaggregated classification, I sum Producer/Merchant, Swap Dealer, Managed Money, Other Reportables, Non-Reportables, and the spreading components actually present in the WRDS feed (MM, OR, SW).

Table 3. COT accounting-identity checks

Check	Share within 1 contract	Maximum long–short discrepancy
Legacy long = total OI	100.0%	0 contracts
Legacy short = total OI	100.0%	0 contracts
Disaggregated long = total OI	100.0%	0 contracts
Disaggregated short = total OI	100.0%	0 contracts

Here, `openinterest` on each category/side row is the number of outstanding contracts attributed to that COT category, while `TO_T` is aggregate market open interest. The COT categories therefore partition outstanding open interest rather than measuring trading volume.

Answer. Both accounting identities hold in 100.0% of weeks to within one contract. The zero long-versus-short discrepancy provides an additional internal consistency check.

1(b) Managed-Money Net Position, Front-Month Price, and COT Index

$$MM_net = MM_L - MM_S \quad | \quad COT_t = (MM_net,t - \min_{3y} MM_net) / (\max_{3y} MM_net - \min_{3y} MM_net)$$

The nearby CL price series uses one active contract at a time and rolls five observed CL sessions before last trading day. The sample is deliberately truncated at the last contract whose expiry window is fully observed; the code does not invent a future exchange calendar. For return calculations in parts (d) and (e), a separate self-financing roll-consistent index removes contract-level jumps at rolls.

Table 4. Nearby-series and signal-construction audit

Audit item	Result
------------	--------

Audit item	Result
Raw CL price history	1999-11-22 to 2026-04-03
Completed-roll nearby series	2006-06-01 to 2026-03-13
Active sessions / usable settlements	4,982 / 4,980
Contracts used	237
Missing active-contract settlements	2
First valid 3-year COT index	2009-06-02
Price-matched COT snapshots	1,031 / 1,057
Raw MM_net missing reports	4 dates: 2006-06-06, 2018-07-31, 2019-02-26, 2019-03-26
Strict 156-report COT-index audit	155-row warm-up; 191 post-warm-up rows invalid from incomplete windows; 711 valid index rows

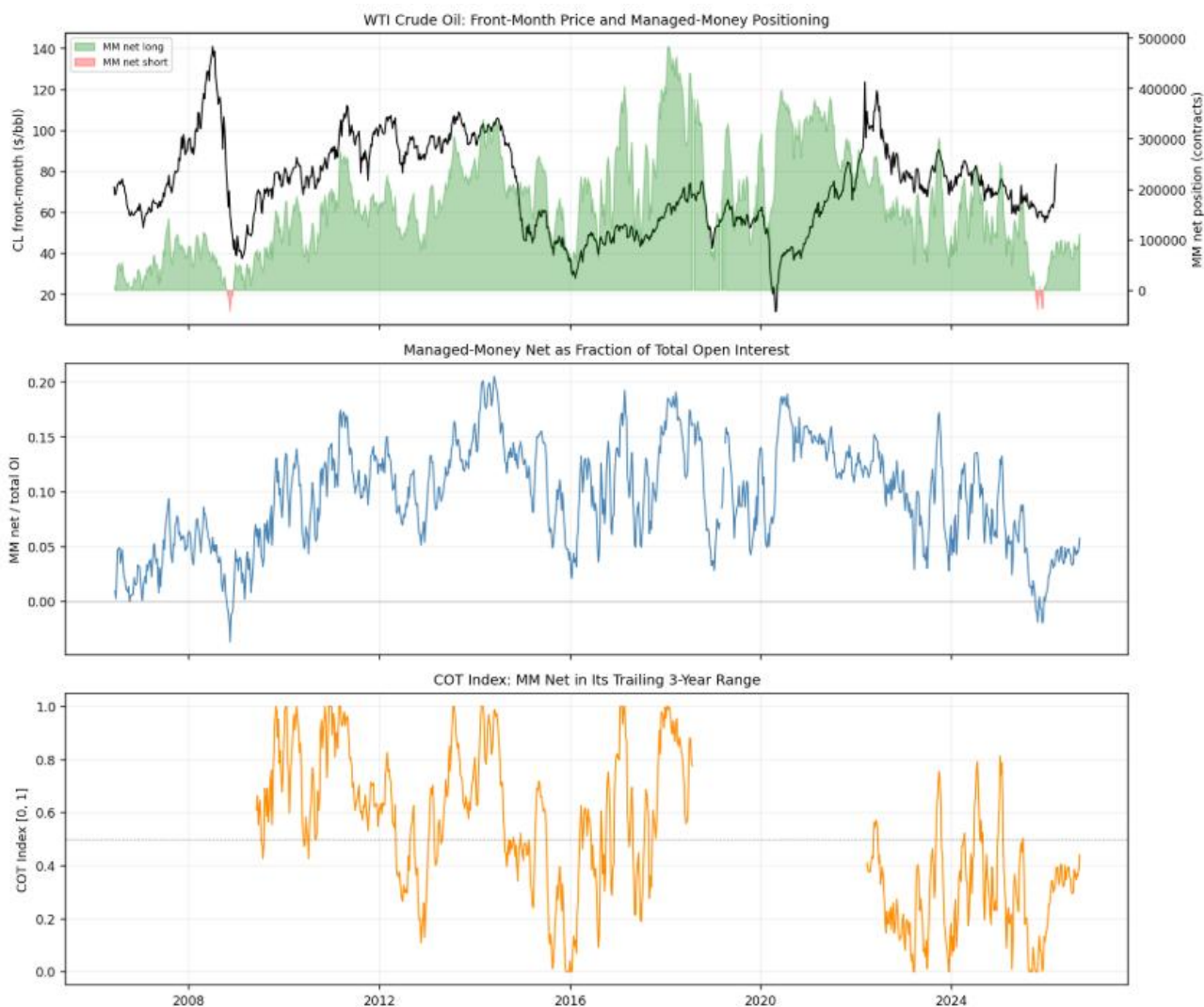


Figure 1. WTI nearby price and managed-money net positioning; MM net normalized by total OI; and the strict trailing three-year COT index.

Managed money is generally net long during stronger crude-price regimes and reduces or flips short during major selloffs, which is consistent with meaningful trend-following exposure within the managed-money category. The figure is descriptive: co-movement does not identify causal direction.

1(c) Commercial Hedging Pressure

$$HP_t = (CM_S - CM_L) / (CM_S + CM_L)$$

Table 5. Commercial hedging-pressure summary

Metric	Result
Weeks with HP > 0 (commercials net short)	96.7%
Mean HP	0.1575
Median HP	0.1674

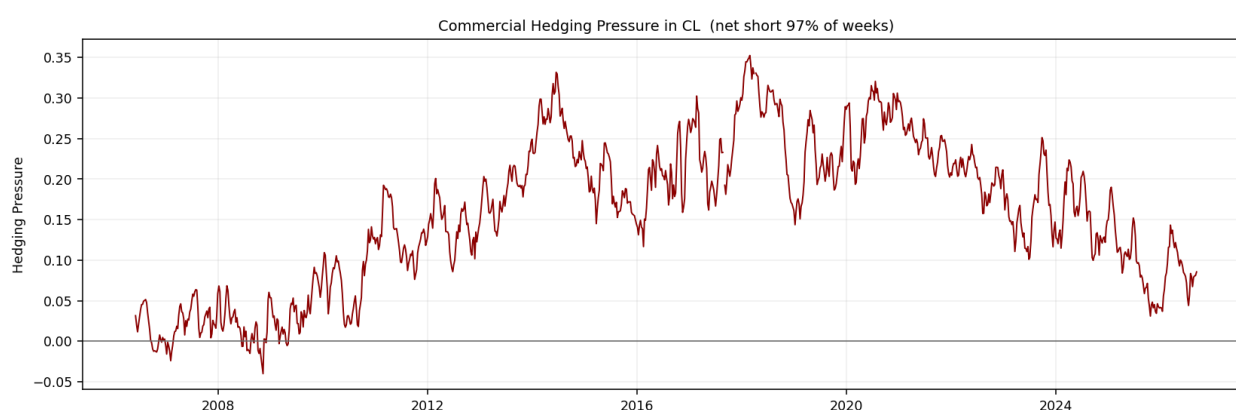


Figure 2. Commercial hedging pressure in CL. Positive HP denotes a net-short commercial position.

A net-short share near 97% is the empirical signature of Keynes's (1930) *normal backwardation*: commodity producers are natural short hedgers whose aggregate selling exceeds the long hedging of processors and end-users, so the commercial side is structurally net short. Crude producers and merchants commonly sell futures to hedge physical inventory or future production; consumer/refiner hedges operate in the opposite direction but are smaller in aggregate. The legacy commercial category is heterogeneous—especially around the pre-2009 classification transition—so the persistent sign is a positioning fact, not by itself evidence of a risk premium or a causal price effect.

1(d) Contemporaneous Returns and Managed-Money Flow

Tuesday-to-Tuesday returns are measured on the roll-consistent nearby return index, while ΔMM_{net} is the same-week change in managed-money net contracts. This prevents calendar-spread level changes at a contract switch from being misclassified as outright crude returns.

Table 6. Contemporaneous positioning–return relationship

Statistic	Estimate
Observations	1,023
Contemporaneous correlation	+0.3959
Slope	+1,497.2 contracts per +1% CL return
OLS t-stat of slope	+13.78

Statistic	Estimate
R^2	0.1568
HAC t-stat of slope (4 weekly lags)	+2.86

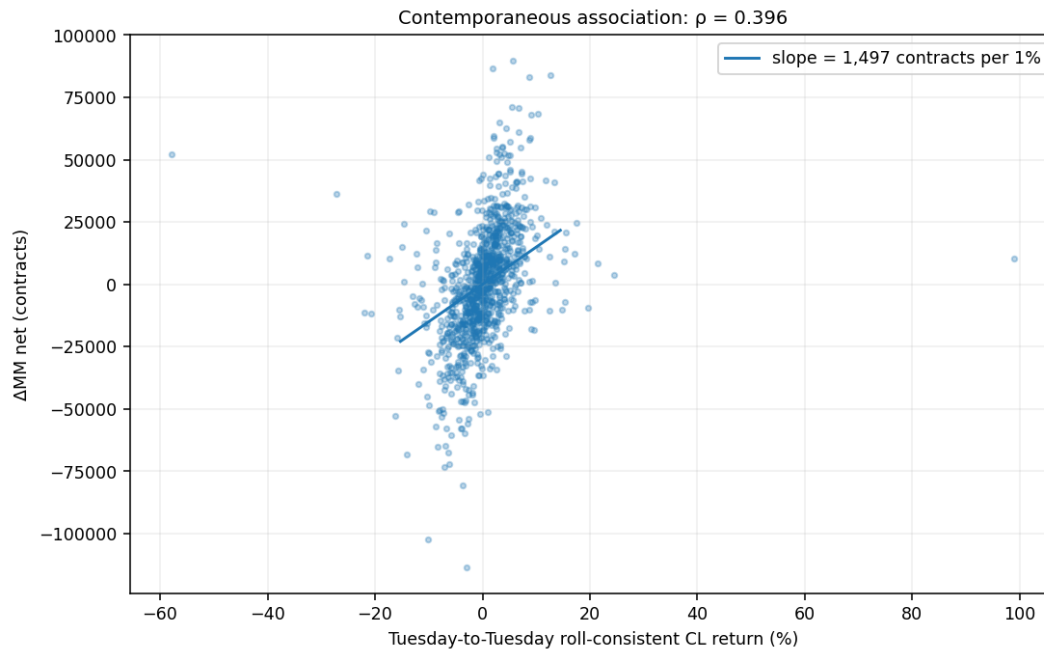


Figure 3. Same-week change in managed-money net position versus roll-consistent CL return.

Interpretation. The dominant relationship is contemporaneous. The same-week correlation is +0.3959; a four-lag HAC diagnostic gives $t = +2.86$. The one-week lead/lag correlations are much smaller: price-leading $\rho = +0.0579$ (HAC $t = +1.34$) and positioning-leading $\rho = +0.0706$ (HAC $t = +1.38$). These lead/lag statistics are descriptive diagnostics, not causal identification. The weekly data therefore look more concurrent than sequential, consistent with within-week adjustment or common information shocks.

1(e) Predictive Regressions

Each COT snapshot is mapped to the following Friday target date. The assignment-convention regressions use the most recent return-index observation within three calendar days of that Friday and exact 1-, 4-, and 13-week calendar horizons. Signals are standardized within each regression sample. I report the requested OLS slope t-statistic and a HAC/Newey-West robustness statistic with $h-1$ lags for h -week overlapping returns. Because the Friday CL settlement precedes the public COT release, I additionally rerun all nine specifications from the first CL settlement strictly after the release Friday.

Table 7. Predictive regressions of forward CL returns on standardized COT signals

Signal	Horizon	n	IC	Slope (% / 1σ)	OLS t	HAC t
COT index	1w	684	-0.0333	-0.149	-0.87	-0.87
COT index	4w	681	-0.0494	-0.418	-1.29	-0.89
COT index	13w	672	-0.1103	-1.532	-2.87	-1.22
Hedging pressure	1w	1,028	-0.0074	-0.040	-0.24	-0.24
Hedging pressure	4w	1,025	-0.0031	-0.034	-0.10	-0.06
Hedging pressure	13w	1,016	-0.0167	-0.331	-0.53	-0.15
MM net / OI	1w	1,026	+0.0140	+0.075	+0.45	+0.43
MM net / OI	4w	1,023	+0.0283	+0.313	+0.91	+0.52
MM net / OI	13w	1,014	-0.0303	-0.603	-0.96	-0.30

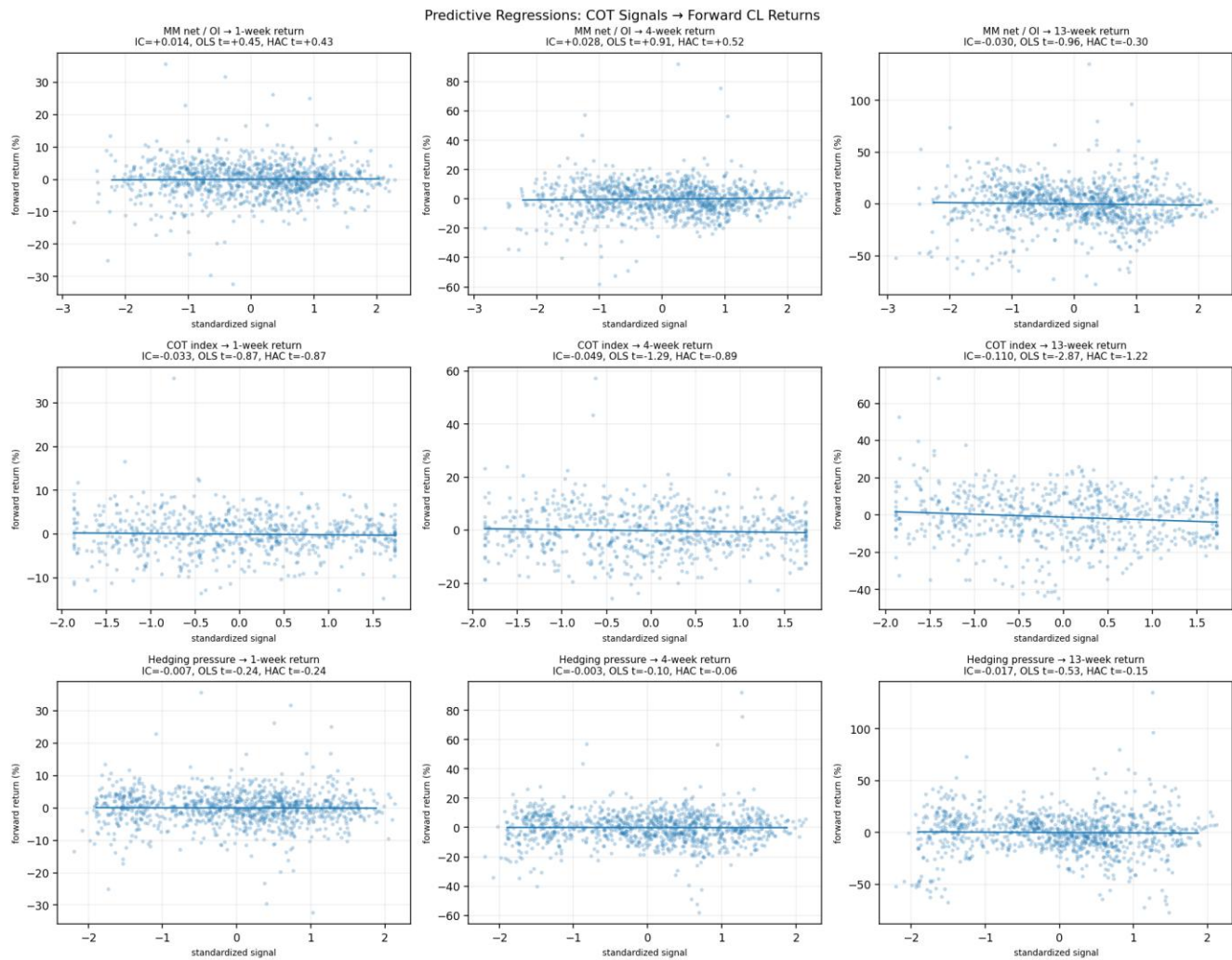


Figure 4. Predictive regressions at 1-, 4-, and 13-week horizons. Panel titles report IC, OLS t , and HAC t .

The predictive evidence is weak under both timing conventions. Under the assignment convention, the clearest case is the 13-week COT-index regression: OLS gives $t = -2.87$, but HAC falls to -1.22 once overlapping-return serial dependence is allowed for. Starting instead from the first settlement strictly after the release Friday gives a very similar 13-week result (IC = -0.1146 , HAC $t = -1.27$). Across all nine post-release specifications, the largest absolute HAC t -statistic is 1.27. The timing correction therefore does not change the conclusion: none of the signals shows strong robust predictability.

Table 8. Forward-return alignment audit

Horizon	Usable returns	Last start target	Last end observation	Max match age
1 week	1,030	2026-03-06	2026-03-13	2 days
4 weeks	1,027	2026-02-13	2026-03-13	2 days
13 weeks	1,018	2025-12-12	2026-03-13	2 days

Table 7A. Timing robustness: assignment Friday start versus first post-release settlement

Signal	Horizon	IC Friday	HAC t Friday	IC post-release	HAC t post-release
COT index	1w	-0.0333	-0.87	-0.0467	-1.19
COT index	4w	-0.0494	-0.89	-0.0450	-0.80
COT index	13w	-0.1103	-1.22	-0.1146	-1.27
Hedging pressure	1w	-0.0074	-0.24	-0.0019	-0.06
Hedging pressure	4w	-0.0031	-0.06	+0.0055	+0.10
Hedging pressure	13w	-0.0167	-0.15	-0.0134	-0.12

Signal	Horizon	IC Friday	HAC t Friday	IC post-release	HAC t post-release
MM net / OI	1w	+0.0140	+0.43	+0.0086	+0.30
MM net / OI	4w	+0.0283	+0.52	+0.0289	+0.57
MM net / OI	13w	−0.0303	−0.30	−0.0375	−0.38

Risk-control note. The largest raw-versus-roll return adjustment is about 34.45 percentage points around the April 2020 transition. Removing such contract-switch level jumps is essential: otherwise roll structure can contaminate both the contemporaneous and predictive results.

Statistical predictability is distinct from an implementable trading strategy. The post-release daily-settlement robustness check removes the most direct Friday-settlement look-ahead concern, but transaction costs, intraday release execution, margin/capital conventions, roll execution, and out-of-sample stability remain separate questions. Note on signal standardization: the regressions standardize each signal by its full-sample mean and standard deviation. This does not affect the t-statistics, ICs, or any inference reported above — it only rescales the slope coefficient — but an expanding-window z-score (using only data available at each forecast date) would be more rigorous for a predictive framing.

Question 2 — Seasonal Calendar Spreads into Expiry

Replication repository: Full executable notebook, supporting code, and environment requirements are available on [GitHub](https://github.com/ajo2159/IEOR-E4576-HW2) (<https://github.com/ajo2159/IEOR-E4576-HW2>).

Each spread is aligned on trading days to the front leg's LTD/FND convention and rebased to zero at the start of a common $(-120, 0)$ window. The instructor-supplied futsread package is used unchanged. For the reported assignment results, the notebook explicitly sets `min_coverage = 1.0` and asserts that every retained annual instance has all 121 offsets from -120 through 0 . The overlay shows individual annual instances; the condensed panel shows the cross-year mean and median with a ± 1 s.e. band. "Scenario agreement" is the fraction of these strict full-window annual P&Ls with the sign predicted by the seasonal story.

Table 9. Spread definitions

Spread	Long leg	Short leg	Expected sign	Full-window n
NG[H][J0]	March NG	April NG	Down	27
ZC[U][Z0]	September corn	December corn	Down	26
ZW[K][N0]	May SRW wheat	July SRW wheat	Down	26
HE[V][Z0]	October lean hogs	December lean hogs	Up	25

Common inference note. For every spread, futsread reports $t = \text{mean(P\&L)} / [\text{sd(P\&L)} / \sqrt{n}]$ across annual instances. The notebook additionally enforces exact 121-point $(-120, 0)$ coverage at the call site while leaving the instructor package unchanged. This is a legitimate one-sample test under its assumptions, but it does not establish causality, after-cost profitability, or out-of-sample persistence.

2(a) Natural Gas — NG[H][J0]

Long March natural gas and short April of the same contract year. March is the last withdrawal-season contract; the seasonal hypothesis is that the end-of-winter scarcity premium erodes as winter passes without a storage crisis, so March/April sells off into March expiry.

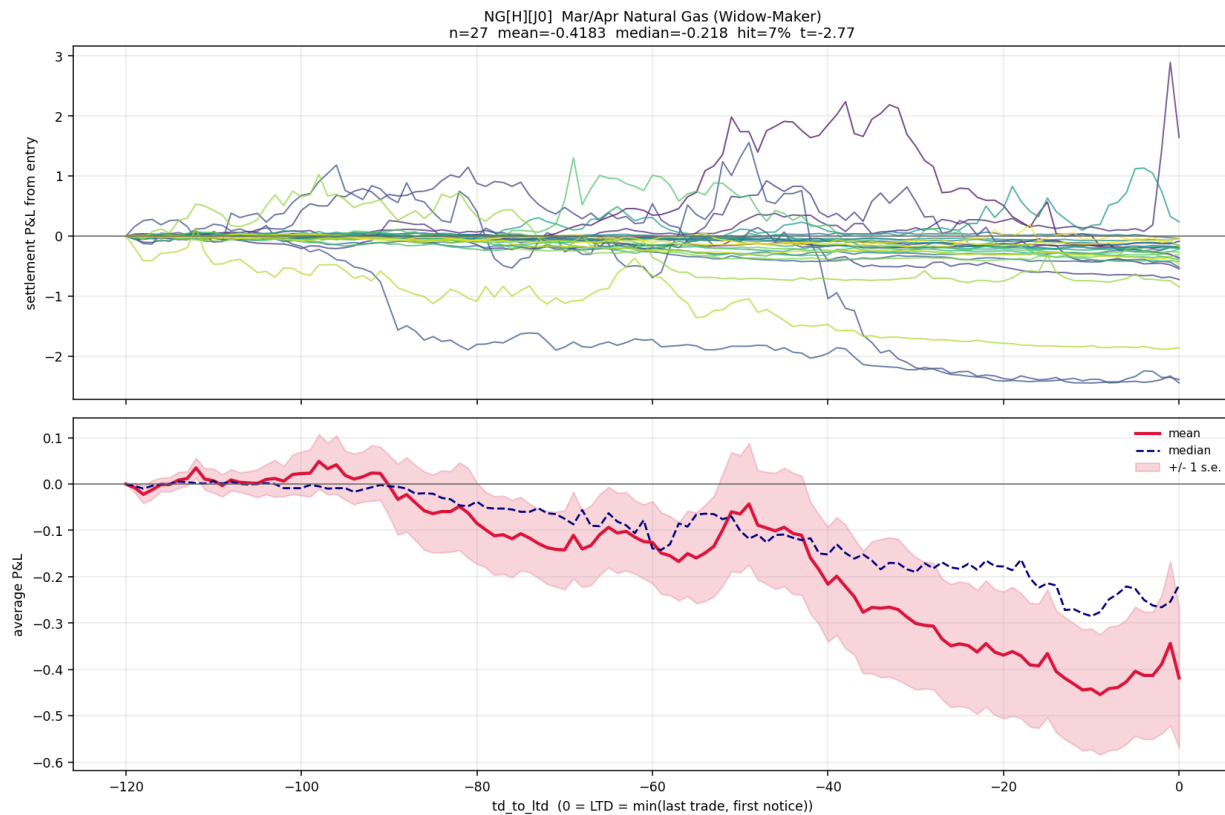


Figure 5. NG March/Apr (“widow-maker”) annual overlays and condensed average over $(-120, 0)$.

Table 10. NG[H][J0] summary

n	Mean	Median	Agreement	Opposite-direction years	t
27	-0.4183	-0.2180	25/27 = 92.6%	2003, 2014	-2.77

The mean path is negative and the decline builds steadily from roughly -60 trading days into expiry, matching the seasonal mechanism. The two positive exceptions are 2003 and 2014. Those episodes are consistent with winter scarcity remaining elevated unusually late; 2014 in particular coincided with the severe polar-vortex winter. This is historical context, not causal identification from the spread path itself.

The reported t-statistic is the ordinary one-sample statistic across annual window P&Ls. Under its assumptions it tests whether the mean annual P&L is zero; it does not establish causality, after-cost profitability, robustness to a different window, or future persistence.

2(b) Corn — ZC[U][Z0]

Long September corn and short December of the same contract year. September is the last old-crop contract and December prices the incoming harvest, so the hypothesis is that old-crop scarcity value is competed away as harvest approaches.

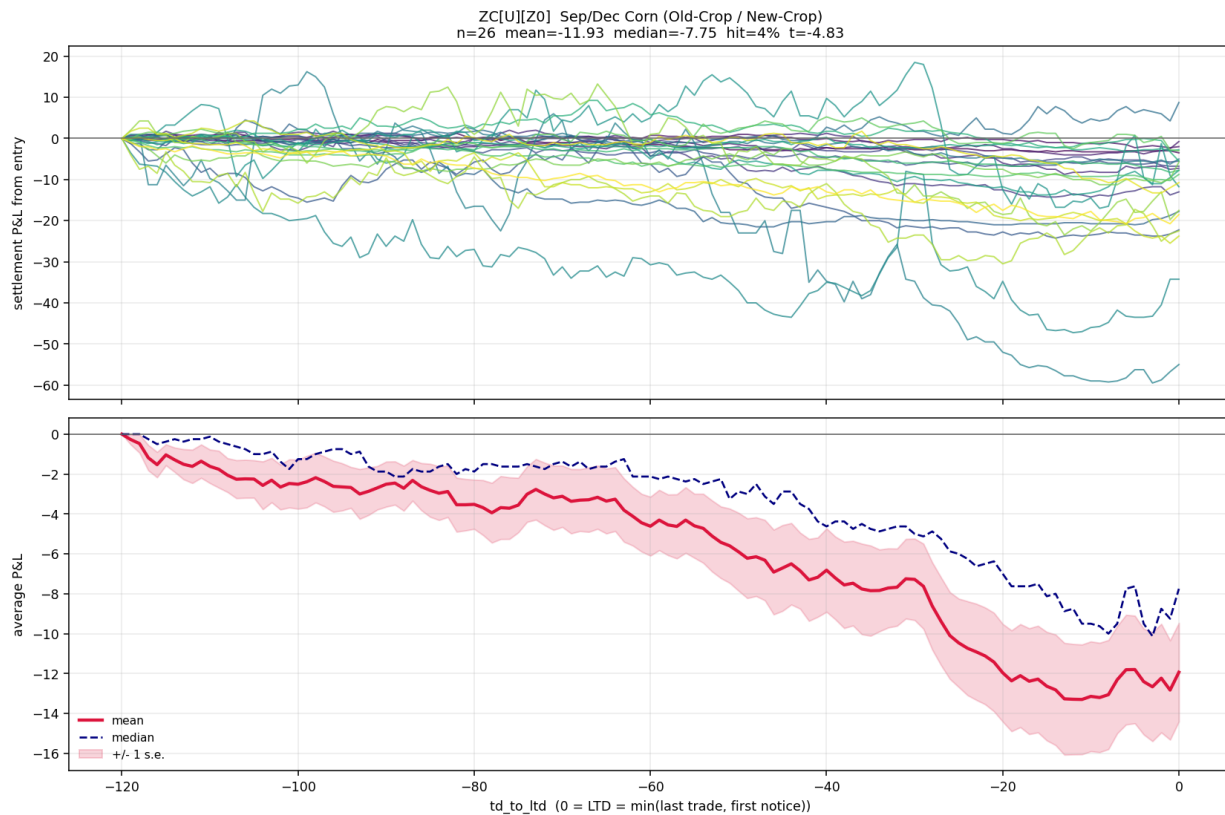


Figure 6. ZC September/December annual overlays and condensed average over $(-120, 0)$.

Table 11. ZC[U][Z0] summary

n	Mean	Median	Agreement	Opposite-direction years	t
26	-11.9327	-7.7500	25/26 = 96.2%	2009	-4.83

The average decline is economically large and becomes especially pronounced inside roughly the final 40 trading days. The only positive exception is 2009. USDA reporting that season described materially delayed crop development and harvest well behind normal by early autumn, which is consistent with new-crop supply arriving later than usual and supporting the nearby old-crop leg.

The t-statistic of -4.83 indicates a sample mean large relative to its cross-year standard error under the one-sample assumptions. It is not, by itself, evidence of a structural law, trading profitability after costs, or out-of-sample persistence.

Historical context: USDA NASS, Crop Production, October 2009.

2(c) Wheat — ZW[K][N0]

Long May SRW wheat and short July of the same contract year. The mechanism is the same old-crop/new-crop transition as corn, but it occurs earlier because winter wheat is harvested in June.

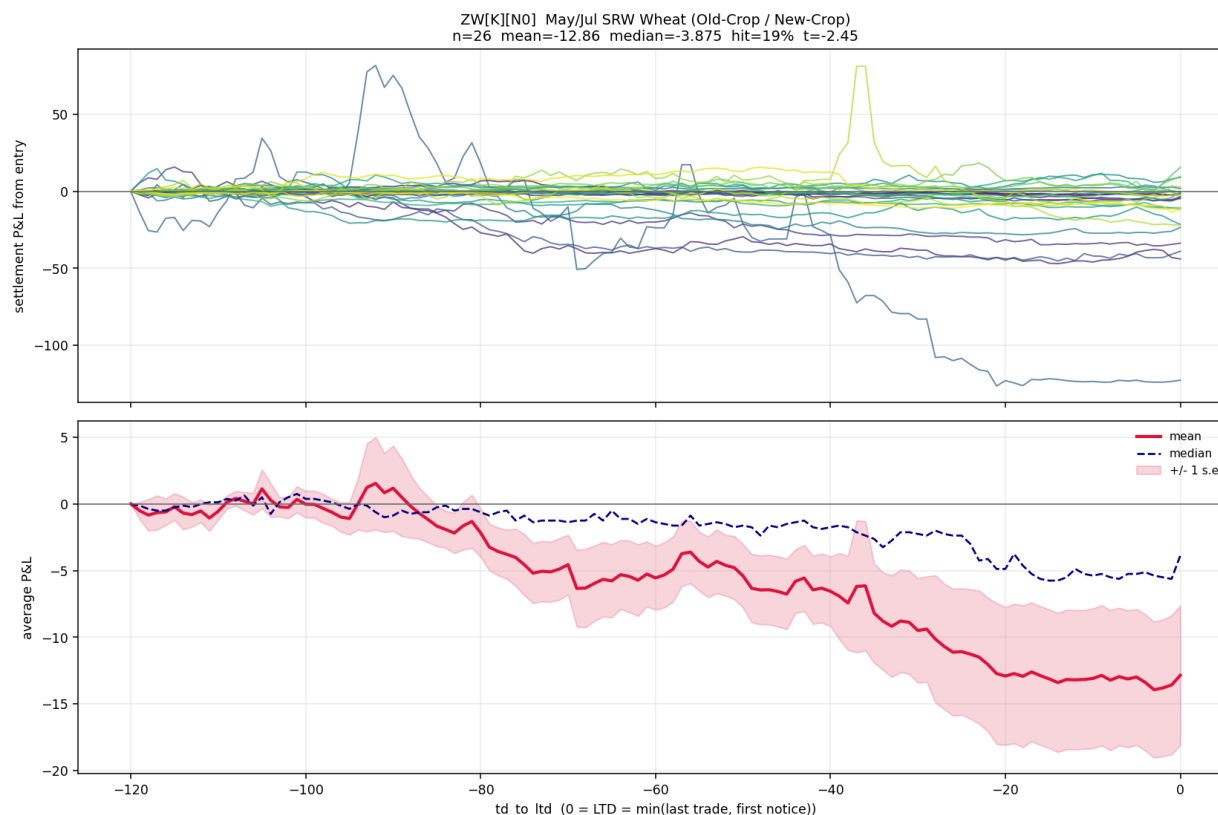


Figure 7. ZW May/July annual overlays and condensed average over $(-120, 0)$.

Table 12. ZW[K][N0] summary

n	Mean	Median	Agreement	Opposite-direction years	t
26	-12.8558	-3.8750	21/26 = 80.8%	2009, 2012, 2018, 2020, 2021	-2.45

The mean/median gap (-12.86 versus -3.88) indicates substantial left skew: a few very negative years pull the average well below the typical outcome. The positive exceptions cluster into plausible tight-supply or crop-stress settings. 2009, 2012, and 2021 were associated with tighter wheat balance sheets or stronger old-crop use; 2018 coincided with substantial drought concern and a smaller projected crop; and 2020 coincided with the COVID-era surge in U.S. retail flour and wheat-product demand documented by USDA. These mechanisms are consistent with May retaining more relative value than the normal seasonal pattern implies, but they are not identified causally by the spread backtest.

Historical context: USDA ERS, Rising Wheat Prices Outpace Input Costs (2009); Wheat Outlook (June 2012, May 2018, May 2020, May 2021).

2(d) Lean Hogs — HE[V][Z0]

Long October lean hogs and short December. Unlike the physically delivered examples above, lean hogs are cash-settled to the CME Lean Hog Index, so the economically important convergence is expected to be concentrated close to expiry.

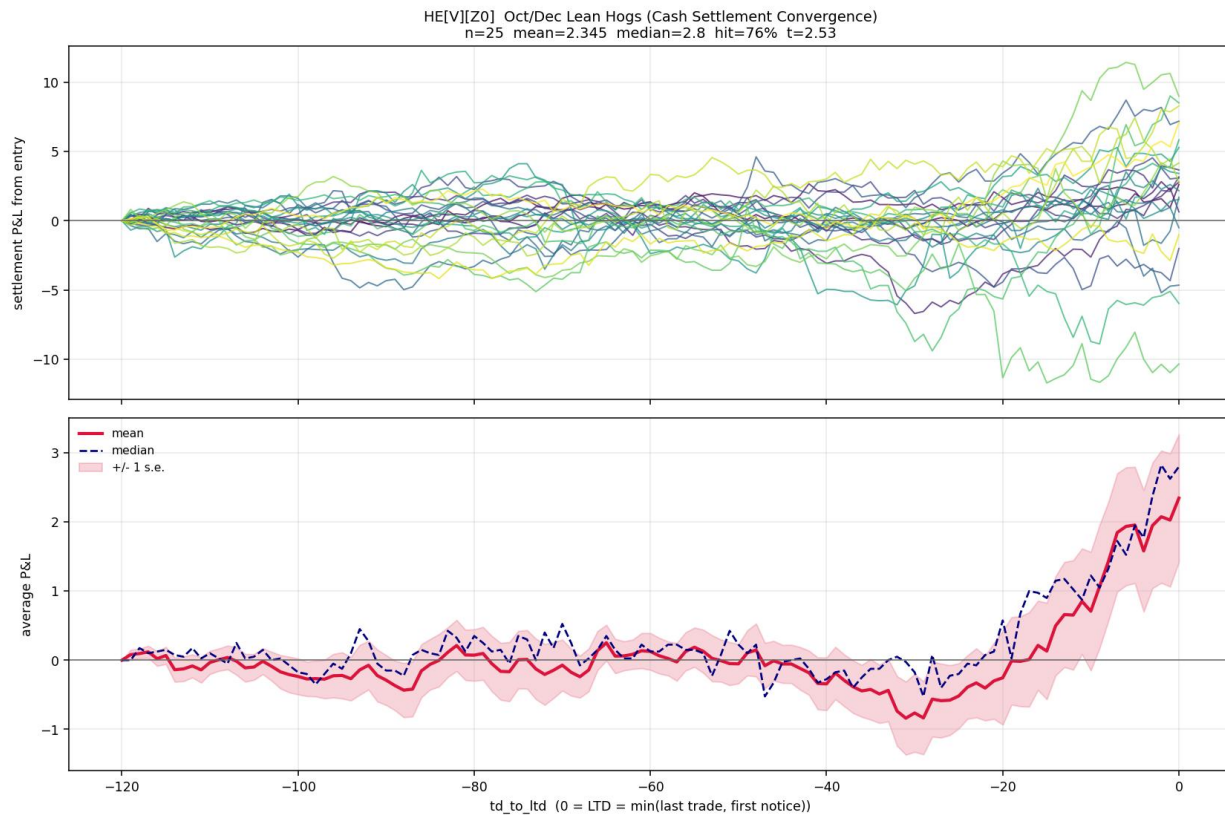


Figure 8. HE October/December annual overlays and condensed average over $(-120, 0)$.

Table 13. HE[V][Z0] summary

n	Mean	Median	Agreement	Opposite-direction years	t
25	+2.3450	+2.8000	19/25 = 76.0%	2007, 2009, 2011, 2017, 2019, 2024	+2.53

The long-run portion of the average path is close to flat: mean P&L is -0.255 at -20 sessions, $+0.709$ at -10 , and $+2.345$ at expiry. This concentration of the move in the final two weeks is the distinctive cash-settlement pattern requested in the question.

The six negative exceptions have different fundamental backdrops. 2007 featured accelerating second-half pork supply; 2009 combined recession-related demand weakness with H1N1-related trade disruption; 2011 mixed larger fall hog supplies with strong Asian export demand; 2017 occurred amid record-production projections; 2019 followed the African-swine-fever shock to Chinese pork supply and rapidly rising import expectations; and 2024 featured heavier weights, higher production, and weaker wholesale pork prices. These facts provide contemporaneous mechanisms consistent with the exceptions but do not prove their causes.

The t-statistic is again a proper one-sample statistic across annual P&Ls. The inferential sample is 25 annual instances—not the roughly 120 daily points shown in each path—and the result does not by itself establish an after-cost trading strategy. With 6 exceptions out of 25 instances (76% agreement) and $t = +2.53$, this is the weakest result of the four spreads. The 95% confidence interval for the mean annual P&L extends close to zero, so the inference is more sensitive to the inclusion or exclusion of individual years than for the corn or natural-gas cases.

Historical context: USDA ERS Livestock, Dairy, and Poultry Outlook (March 2007; September/November 2011; September 2024); USDA ERS, China's Volatile Pork Industry (H1N1/2009); USDA livestock outlook for 2017; USDA FAS, Pork 2019 Export Highlights.

Data Integrity, Reproducibility, and Source Notes

The final notebook executes sequentially with no saved exceptions and concludes with explicit assertions on duplicate dates, COT-index bounds, bounded price-match ages, monotonic return-index dates, roll-anchor timing, forward-return endpoint ordering, post-release timing, and exact 121-point seasonal-spread windows. Question 1 source snapshots are frozen at COT $\leq$ 2026-09-08 and CL settlements $\leq$ 2026-04-03, with SHA256 hashes recorded in the notebook. Question 2 uses the instructor-supplied futsread package unchanged and records the resulting combo date spans and terminal-date source audit.

Table 14. Cross-check between the final notebook and this report

Item	Report / notebook consistency
COT accounting	100.0% identity checks and 0-contract long–short discrepancy match the notebook output.
Front-month construction	237 contracts, 4,982 active sessions, 4,980 usable settlements, and 2 missing active settlements match the run.
Contemporaneous regression	$\rho = +0.3959$, slope = +1,497.2 contracts per 1%, $t = +13.78$, $R^2 = 0.1568$ match the run.
Predictive regressions	All 9 assignment-timing rows reproduce exactly; post-release robustness gives largest $ HAC\ t = 1.27$ and leaves the conclusion unchanged.
Seasonal spreads	n, mean, median, sign-agreement rate, exception years, and t-stat match each futsread output.
COT-index attrition audit	4 raw MM_net missing reports, 155-report warm-up, 191 post-warm-up incomplete windows, and 711 valid index rows match the run.
Strict Q2 windows	NG 27, ZC 26, ZW 26, HE 25 retained instances each pass exact 121-offset $[-120,0]$ completeness assertions.
Q1 snapshot hashes	COT SHA256 967457209b6c...; CL-settlement SHA256 e1e6fe9e06b... recorded with row counts and date spans.

Historical context sources cited in Question 2

- USDA NASS. Crop Production, October 2009.
- USDA ERS. Rising Wheat Prices Outpace Input Costs (2009).
- USDA ERS. Wheat Outlook, June 2012; May 2018; May 2020; May 2021.
- USDA ERS. Livestock, Dairy, and Poultry Outlook, March 2007; September/November 2011; September 2024.
- USDA ERS. China's Volatile Pork Industry (discussion of H1N1-era 2009 trade disruption).
- USDA livestock outlook material for 2017; USDA FAS, Pork 2019 Export Highlights.

Reproducibility. The complete executed notebook is the authoritative code artifact. It records the Python/package versions, frozen Question 1 WRDS snapshot dates, source hashes, and the instructor futsread package path/signature used for Question 2. The appendix below reproduces the final notebook code cells in execution order; numerical outputs remain summarized in the main report rather than duplicated verbatim.

Appendix A — Complete Python Code

The following code is reproduced from the final executed notebook in execution order. Markdown-only narrative cells are omitted because the corresponding analysis is presented in the main report.

Code cell 1

```
from pathlib import Path
import hashlib
import importlib
import inspect
import importlib.metadata as importlib_metadata
import os
import platform
import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats as sp_stats
import statsmodels.api as sm

plt.rcParams.update({
    'figure.dpi': 140,
    'savefig.dpi': 140,
    'font.size': 9,
    'axes.titlesize': 10,
})

# Freeze the submitted Question 1 data snapshot. These caps apply to the
# student's direct WRDS COT/CL queries only; the instructor-supplied
# futspeed package used in Question 2 is intentionally left unmodified.
COT_AS_OF_DATE = pd.Timestamp('2026-09-08')
CL_AS_OF_DATE = pd.Timestamp('2026-04-03')
PYTHON_TARGET = '3.13.9'

print(f'Python runtime: {platform.python_version()} (submission target {PYTHON_TARGET})')
print(f'COT snapshot cap: {COT_AS_OF_DATE:%Y-%m-%d}')
print(f'Q1 CL snapshot cap: {CL_AS_OF_DATE:%Y-%m-%d}')
print('Core package versions:')
for pkg in ['numpy', 'pandas', 'matplotlib', 'scipy', 'statsmodels', 'wrds', 'pyarrow']:
    try:
        print(f' {pkg}<12s} {importlib_metadata.version(pkg)}')
    except importlib_metadata.PackageNotFoundError:
        print(f' {pkg}<12s} NOT INSTALLED')

# Resolve the WRDS username explicitly so the wrds client never falls back to
# a stale username from a local config or previous session.
WRDS_USER = os.environ.get('WRDS_USERNAME', '').strip()
if not WRDS_USER:
    WRDS_USER = input('Enter your WRDS username (e.g., ajo2159): ').strip()
if not WRDS_USER:
    raise ValueError('A WRDS username is required.')

print(f'Using WRDS username: {WRDS_USER}')
```

Code cell 2

```
import wrds

conn = wrds.Connection(wrds_username=WRDS_USER)

cot_raw = conn.raw_sql(f"""
    SELECT date, dsmnem, openinterest, numtrades
    FROM trdstm.dsfcotcrepval
    WHERE contrcode = 1986
    AND date >= '2006-06-01'
    AND date <= '{COT_AS_OF_DATE:%Y-%m-%d}'
    ORDER BY date, dsmnem
""", date_cols=['date'])

assert not cot_raw.empty
assert cot_raw['date'].notna().all()
assert cot_raw['dsmnem'].notna().all()
assert cot_raw['date'].max() <= COT_AS_OF_DATE

# Compact non-redistributive fingerprint for snapshot auditing.
cot_hash = pd.util.hash_pandas_object(
    cot_raw[['date', 'dsmnem', 'openinterest', 'numtrades']], index=False
).values.tobytes()
COT_SHA256 = hashlib.sha256(cot_hash).hexdigest()

print(f'COT rows: {len(cot_raw):,}')
print(f'Date range: {cot_raw["date"].min():%Y-%m-%d} to {cot_raw["date"].max():%Y-%m-%d}')
print(f'Unique report types: {cot_raw["dsmnem"].nunique()}')
print(f'COT snapshot SHA256: {COT_SHA256}')
print('Report-date weekdays:')
```

```
print(cot_raw[['date_']].drop_duplicates()['date_'].dt.day_name().value_counts().to_string())
cot_raw.head()
```

Code cell 3

```
# Last four characters: category(2) + side(1) + basis(1).
cot_raw['suffix'] = cot_raw['dsmnem'].str[-4:]
cot_raw['category'] = cot_raw['suffix'].str[:2]
cot_raw['side'] = cot_raw['suffix'].str[2]
cot_raw['basis'] = cot_raw['suffix'].str[3]

# The assignment asks for futures-only observations.
cot = cot_raw.loc[cot_raw['basis'].eq('F')].copy()
cot['col'] = cot['category'] + '_' + cot['side']

# A pivot with aggfunc='first' can silently hide duplicates, so check first.
dup = cot.duplicated(['date_', 'col'], keep=False)
if dup.any():
    bad = cot.loc[dup, ['date_', 'dsmnem', 'col', 'openinterest']]
    raise ValueError(f'Duplicate futures-only COT keys detected:\n{bad.head(20)}')

print(f'Futures-only rows: {len(cot):,}')
print('Categories:', sorted(cot['category'].unique()))
print('Sides:', sorted(cot['side'].unique()))
```

Code cell 4

```
# One row per report date, one column per category/side.
wide = (cot.pivot(index='date_', columns='col', values='openinterest')
        .sort_index())

assert wide.index.is_monotonic_increasing
assert not wide.index.duplicated().any()

print(f'Pivoted: {len(wide)} weeks, {wide.shape[1]} columns')
wide.head()
```

Code cell 5

```
# — Verify the LEGACY accounting identity —
# OI = CM_L + NC_L + NC_X + NR_L (long side)
# OI = CM_S + NC_S + NC_X + NR_S (short side)

legacy_long = wide['CM_L'] + wide['NC_L'] + wide['NC_X'] + wide['NR_L']
legacy_short = wide['CM_S'] + wide['NC_S'] + wide['NC_X'] + wide['NR_S']

# Total OI: WRDS encodes total OI as category TO, side T (a single number,
# not split by long/short). TO_T is the column to use.
if 'TO_T' in wide.columns:
    total_oi = wide['TO_T']
elif 'TO_L' in wide.columns:
    total_oi = wide['TO_L']
else:
    total_oi = legacy_long # use the identity itself as reference
print('Note: no TOT column found; checking long-side = short-side only.')

err_long = (legacy_long - total_oi).abs()
err_short = (legacy_short - total_oi).abs()
err_ls = (legacy_long - legacy_short).abs() # long vs short cross-check

frac_long = (err_long <= 1).mean()
frac_short = (err_short <= 1).mean()
frac_ls = (err_ls <= 1).mean()

print('Legacy accounting identity:')
print(f' Long side = Total OI within 1 contract: {frac_long:.1%}')
print(f' Short side = Total OI within 1 contract: {frac_short:.1%}')
print(f' Long side = Short side within 1 contract: {frac_ls:.1%}')
print(f' Max |long - short| discrepancy: {err_ls.max():.0f} contracts')
```

Code cell 6

```
# — Verify the DISAGGREGATED accounting identity —
# Disaggregated categories: PM, SW, MM, OR (plus NR as residual)
# Each reportable category has Long, Short, and Spreading columns.
# OI = PM_L + SW_L + MM_L + OR_L + NR_L + (all Spreading terms)

# Collect whatever spreading columns exist
spread_cols = [c for c in wide.columns if c.endswith('_X') and c[:2] in ('PM', 'SW', 'MM', 'OR')]
print(f'Spreading columns found: {spread_cols}')

disagg_long = wide['PM_L'] + wide['SW_L'] + wide['MM_L'] + wide['OR_L'] + wide['NR_L']
disagg_short = wide['PM_S'] + wide['SW_S'] + wide['MM_S'] + wide['OR_S'] + wide['NR_S']
for sc in spread_cols:
    disagg_long = disagg_long + wide[sc]
    disagg_short = disagg_short + wide[sc]
```

```

err_dl = (disagg_long - total_oi).abs()
err_ds = (disagg_short - total_oi).abs()
err_disagg_ls = (disagg_long - disagg_short).abs()

frac_dl = (err_dl <= 1).mean()
frac_ds = (err_ds <= 1).mean()
frac_disagg_ls = (err_disagg_ls <= 1).mean()

print("\nDisaggregated accounting identity:")
print(f' Long side = Total OI within 1 contract: {frac_dl:.1%}')
print(f' Short side = Total OI within 1 contract: {frac_ds:.1%}')
print(f' Long side = Short side within 1 contract: {frac_disagg_ls:.1%}')
print(f' Max |long - short| discrepancy: {err_disagg_ls.max():.0f} contracts')
```

Code cell 7

```

# Contract metadata for CL. The assignment specifies rolling a few days before
# each contract's last trading day, so lasttrddate is the roll anchor here.
# For Question 1 only, startdate is capped to the submitted CL snapshot so
# later listings cannot enter a future rerun.
cl_info = conn.raw_sql(f"""
SELECT futcode, contrcode, contrdate, lasttrddate, firstnoticedate, startdate
FROM trdstm.wrds contract_info
WHERE contrcode = 1986
AND lasttrddate >= '2006-01-01'
AND (startdate IS NULL OR startdate <= '{CL_AS_OF_DATE:%Y-%m-%d}')
ORDER BY lasttrddate, futcode
""", date_cols=['lasttrddate', 'firstnoticedate', 'startdate'])

cl_info['futcode'] = cl_info['futcode'].astype('int64')
cl_info = (cl_info.dropna(subset=['lasttrddate'])
          .sort_values(['lasttrddate', 'futcode'])
          .drop_duplicates('futcode'))

assert cl_info['lasttrddate'].is_monotonic_increasing
print(f'CL contracts listed by snapshot date: {len(cl_info)}')
cl_info.head()
```

Code cell 8

```

# Daily settlements for all CL contracts in the metadata table, frozen to the
# submitted Question 1 CL snapshot.
futcodes = ','.join(cl_info['futcode'].astype(str))
cl_px = conn.raw_sql(f"""
SELECT DISTINCT futcode, date AS date, settlement
FROM trdstm.wrds fut_contract
WHERE futcode IN ({futcodes})
AND date <= '{CL_AS_OF_DATE:%Y-%m-%d}'
ORDER BY futcode, date_
""", date_cols=['date'])

cl_px['futcode'] = cl_px['futcode'].astype('int64')
cl_px = (cl_px.dropna(subset=['date', 'settlement'])
        .drop_duplicates(['futcode', 'date'])
        .sort_values(['futcode', 'date'])
        .reset_index(drop=True))

assert not cl_px.duplicated(['futcode', 'date']).any()
assert cl_px['date'].max() <= CL_AS_OF_DATE
_cl_hash = pd.util.hash_pandas_object(
    cl_px[['futcode', 'date', 'settlement']], index=False
).values.tobytes()
CL_PRICE_SHA256 = hashlib.sha256(_cl_hash).hexdigest()

print(f'CL price bars: {len(cl_px):,}')
print(f'Price history: {cl_px["date"].min():%Y-%m-%d} to {cl_px["date"].max():%Y-%m-%d}')
print(f'CL settlement snapshot SHA256: {CL_PRICE_SHA256}')
```

Code cell 9

```

def build_front_month(info, prices, roll_sessions=5, start='2006-06-01'):
    """Construct a non-overlapping nearby CL settlement series.

    Each historical contract rolls `roll_sessions` observed CL trading sessions
    before its last trading day. The sample is deliberately truncated at the
    last contract whose LTD is fully observed in the price history; the code
    does not guess a future exchange-session calendar for a live contract.
    """
    if roll_sessions < 1:
        raise ValueError('roll_sessions must be positive')

    info = info.sort_values(['lasttrddate', 'futcode']).copy()
    prices = prices.sort_values(['date', 'futcode']).copy()
    start = pd.Timestamp(start)
    market_end = prices['date'].max()

    # Union of observed CL dates across contracts: realized historical session
    # calendar. Only contracts whose LTD is inside this calendar are eligible.
```

```

sessions = pd.DatetimeIndex(prices['date'].drop_duplicates().sort_values())
completed = info.loc[info['lasttrddate'].le(market_end)].copy()

pieces, audit = [], []
previous_roll = start - pd.Timedelta(nanoseconds=1)

for row in completed.itertuples(index=False):
    # Last observed CL session on or before the contractual LTD.
    ltd_pos = sessions.searchsorted(row.lasttrddate, side='right') - 1
    if ltd_pos < roll_sessions:
        continue

    ltd_session = sessions[ltd_pos]
    roll_date = sessions[ltd_pos - roll_sessions]
    if roll_date <= previous_roll:
        continue

    interval = sessions[(sessions > previous_roll) &
                        (sessions <= roll_date) &
                        (sessions >= start)]
    if len(interval) == 0:
        previous_roll = roll_date
        continue

    contract_px = (prices.loc[prices['futcode'].eq(row.futcode),
                             ['date', 'settlement']]
                  .drop_duplicates('date')
                  .set_index('date')['settlement'])

    piece = pd.DataFrame(index=interval)
    piece.index.name = 'date'
    piece['settlement'] = contract_px.reindex(interval)
    piece['futcode'] = row.futcode
    piece['contrdate'] = row.contrdate
    piece['lasttrddate'] = row.lasttrddate
    piece['ltd_session'] = ltd_session
    piece['roll_date'] = roll_date
    pieces.append(piece)

    audit.append({
        'futcode': row.futcode,
        'contrdate': row.contrdate,
        'start': interval.min(),
        'end': interval.max(),
        'lasttrddate': row.lasttrddate,
        'ltd_session': ltd_session,
        'roll_date': roll_date,
        'n_sessions': len(interval),
        'missing_settlements': int(piece['settlement'].isna().sum()),
    })
    previous_roll = roll_date

if not pieces:
    raise ValueError("No completed front-month observations were constructed.")

front = pd.concat(pieces).sort_index()
audit = pd.DataFrame(audit)

# Core invariants: one active contract per observed session and no overlap
# between consecutive holding intervals.
assert front.index.is_monotonic_increasing
assert not front.index.duplicated().any()
assert audit['start'].is_monotonic_increasing
if len(audit) > 1:
    assert (audit['start'].iloc[1:].to_numpy() >
            audit['end'].iloc[:-1].to_numpy()).all()

return front, audit

front, front_audit = build_front_month(
    cl_info, cl_px, roll_sessions=5, start='2006-06-01'
)
front_px = front['settlement'].dropna().copy()

print(f'Raw CL price history ends: {cl_px["date"].max():%Y-%m-%d}')
print(f'Completed-roll front series ends: {front.index.max():%Y-%m-%d}')
print(f'Front-month schedule: {len(front):,} sessions')
print(f'Usable settlements: {len(front_px):,}')
print(f'Contracts used: {front["futcode"].nunique()}')
print(f'Missing active-contract settlements: {front["settlement"].isna().sum()}')
print(f'Tail of roll audit:')
display(front_audit.tail(8))

```

Code cell 10

```

def bounded_asof(target_dates, series, max_age_days=3, value_name='value'):
    """Match each non-null target to the latest prior observation within a hard limit.

```

Null targets are preserved as unmatched rows. This handles short exchange-calendar gaps (for example, a Thursday close for a Good Friday target) without allowing stale observations to propagate across a long gap or beyond the end of the source series.

```
"""
target_index = pd.to_datetime(pd.Index(target_dates))
targets = pd.DataFrame({
    'target_date': target_index,
    '_order': np.arange(len(target_index)),
})
valid_targets = targets.dropna(subset=['target_date']).sort_values('target_date')

obs = (series.dropna()
        .rename(value_name)
        .rename_axis('obs_date')
        .reset_index()
        .sort_values('obs_date'))

if obs['obs_date'].duplicated().any():
    raise ValueError('Source series has duplicate observation dates.')

if valid_targets.empty:
    matched = pd.DataFrame(columns=['_order', 'obs_date', value_name])
else:
    matched = pd.merge_asof(
        valid_targets, obs,
        left_on='target_date', right_on='obs_date',
        direction='backward',
        tolerance=pd.Timedelta(days=max_age_days),
    )
    matched = matched[['_order', 'obs_date', value_name]]

out = targets.merge(matched, on='_order', how='left')
out['age_days'] = (out['target_date'] - out['obs_date']).dt.days
out = out.sort_values('_order').drop(columns='_order').reset_index(drop=True)

valid = out[value_name].notna()
assert (out.loc[valid, 'obs_date'] <= out.loc[valid, 'target_date']).all()
assert (out.loc[valid, 'age_days'].between(0, max_age_days)).all()
return out
```

```
def bounded_forward_after(target_dates, series, max_delay_days=4, value_name='value'):
    """Match each non-null target to the first observation STRICTLY after it.
```

Null targets are preserved as unmatched rows. This is used for the post-COT-release robustness check. With daily settlements, the first price that is certainly observable after a Friday afternoon COT publication is the next settlement, normally Monday (or Tuesday after a Monday holiday).

```
"""
target_index = pd.to_datetime(pd.Index(target_dates))
targets = pd.DataFrame({
    'target_date': target_index,
    '_order': np.arange(len(target_index)),
})
valid_targets = targets.dropna(subset=['target_date']).sort_values('target_date')
obs = (series.dropna()
        .rename(value_name)
        .rename_axis('obs_date')
        .reset_index()
        .sort_values('obs_date'))
if obs['obs_date'].duplicated().any():
    raise ValueError('Source series has duplicate observation dates.')

if valid_targets.empty:
    matched = pd.DataFrame(columns=['_order', 'obs_date', value_name])
else:
    matched = pd.merge_asof(
        valid_targets, obs,
        left_on='target_date', right_on='obs_date',
        direction='forward', allow_exact_matches=False,
        tolerance=pd.Timedelta(days=max_delay_days),
    )
    matched = matched[['_order', 'obs_date', value_name]]

out = targets.merge(matched, on='_order', how='left')
out['delay_days'] = (out['obs_date'] - out['target_date']).dt.days
out = out.sort_values('_order').drop(columns='_order').reset_index(drop=True)
valid = out[value_name].notna()
assert (out.loc[valid, 'obs_date'] > out.loc[valid, 'target_date']).all()
assert (out.loc[valid, 'delay_days'].between(1, max_delay_days)).all()
return out
```

```
def build_roll_return_index(front, prices, base=100.0, max_roll_anchor_age_days=3):
    """Build a self-financing return index from the active CL contract schedule.
```

Within a contract, returns use that contract's settlement-to-settlement price change. At a roll, the first return in the incoming contract is measured from the incoming contract's settlement on the prior *active* observation date. The old/new contract level difference is therefore never counted as return.

A missing active settlement inside one holding interval is bridged by the next observed settlement of the same contract. At a roll, the incoming contract must have an exact settlement on the roll anchor date. If it does not, the function fails rather than silently inventing a roll price.

```

"""
required_front = {'settlement', 'futcode'}
required_prices = {'date', 'futcode', 'settlement'}
if not required_front.issubset(front.columns):
    raise ValueError(f'front is missing {sorted(required_front - set(front.columns))}')
if not required_prices.issubset(prices.columns):
    raise ValueError(f'prices is missing {sorted(required_prices - set(prices.columns))}')

active = (front.dropna(subset=['settlement', 'futcode'])
         .sort_index()
         .copy())
if active.empty:
    raise ValueError('No active front-month settlements are available.')
if active.index.has_duplicates or not active.index.is_monotonic_increasing:
    raise ValueError('Front schedule must have unique, increasing dates.')

px = (prices.dropna(subset=['date', 'futcode', 'settlement'])
     .drop_duplicates(['futcode', 'date'])
     .copy())
if px.duplicated(['futcode', 'date']).any():
    raise ValueError('Duplicate raw contract settlements remain after cleaning.')
px_lookup = px.set_index(['futcode', 'date'])['settlement']

ret = pd.Series(np.nan, index=active.index, name='roll_consistent_return')
roll_rows = []

for i in range(1, len(active)):
    date = active.index[i]
    prev_date = active.index[i - 1]
    code = int(active.iloc[i]['futcode'])
    prev_code = int(active.iloc[i - 1]['futcode'])
    p1 = float(active.iloc[i]['settlement'])

    if code == prev_code:
        # If an active settlement was missing between prev_date and date,
        # this is simply the multi-session return of the same contract.
        p0 = float(active.iloc[i - 1]['settlement'])
        is_roll = False
    else:
        # Roll at the previous observable active close. Require the incoming
        # contract to have a settlement on exactly that same date.
        key = (code, prev_date)
        if key not in px_lookup.index:
            raise ValueError(
                f'Cannot price roll into futcode {code} on {prev_date:%Y-%m-%d}: '
                f'incoming-contract settlement is missing.'
            )
        p0 = float(px_lookup.loc[key])
        is_roll = True

    scheduled_roll = (pd.Timestamp(active.iloc[i - 1]['roll_date'])
                     if 'roll_date' in active.columns else prev_date)
    anchor_age = (scheduled_roll - prev_date).days
    if anchor_age < 0 or anchor_age > max_roll_anchor_age_days:
        raise ValueError(
            f'Roll anchor for futcode {code} is {anchor_age} calendar days '
            f'from scheduled roll date {scheduled_roll:%Y-%m-%d}.'
        )

    if not np.isfinite(p0) or not np.isfinite(p1) or p0 == 0:
        raise ValueError(f'Invalid settlement around {date:%Y-%m-%d}.')

    r = p1 / p0 - 1.0
    if not np.isfinite(r):
        raise ValueError(f'Non-finite return at {date:%Y-%m-%d}.')
    ret.iloc[i] = r

    if is_roll:
        old_level = float(active.iloc[i - 1]['settlement'])
        raw_jump = p1 / old_level - 1.0 if old_level != 0 else np.nan
        roll_rows.append({
            'roll_effective_date': date,
            'anchor_date': prev_date,
            'scheduled_roll_date': scheduled_roll,
            'anchor_age_days': anchor_age,
            'old_futcode': prev_code,
            'new_futcode': code,
            'old_contract_level': old_level,
            'new_contract_anchor': p0,
            'new_contract_level': p1,
            'raw_stitched_return': raw_jump,
            'roll_consistent_return': r,
            'removed_level_jump_pp': (100.0 * (raw_jump - r))
        })

```

```

        if np.isfinite(raw_jump) else np.nan),
    })

if ret.iloc[1:].isna().any():
    raise ValueError("Return index contains an unexplained missing transition.")

# A base-100 index is only a bookkeeping device: ratios of the index equal
# cumulative roll-consistent futures price returns over any date interval.
return_index = base * (1.0 + ret.fillna(0.0).cumprod())
return_index.name = 'roll_return_index'

# Non-roll observations must agree exactly with ordinary active-contract
# price returns. This catches accidental reindexing or contract mismatches.
same_contract = active['futcode'].eq(active['futcode'].shift(1))
raw_active_ret = active['settlement'].pct_change(fill_method=None)
check = same_contract & ret.notna() & raw_active_ret.notna()
if check.any():
    assert np.allclose(
        ret.loc[check], raw_active_ret.loc[check], rtol=1e-12, atol=1e-12
    )

return return_index, ret, pd.DataFrame(roll_rows)

# Return series used in parts (d) and (e). The raw stitched settlement series
# remains the economically interpretable price level used in part (b).
front_return_index, front_daily_ret, roll_return_audit = build_roll_return_index(
    front, cl_px, base=100.0, max_roll_anchor_age_days=3
)

print(f'Roll-consistent return index: {front_return_index.index.min():%Y-%m-%d} '
      f'to {front_return_index.index.max():%Y-%m-%d}')
print(f'Priced contract transitions: {len(roll_return_audit)}')
if not roll_return_audit.empty:
    print('Largest absolute raw-vs-roll return adjustments (percentage points):')
    display(
        roll_return_audit.assign(
            abs_removed=lambda x: x['removed_level_jump_pp'].abs()
        ).nlargest(8, 'abs_removed')[[
            'roll_effective_date', 'anchor_date', 'scheduled_roll_date',
            'old_futcode', 'new_futcode', 'raw_stitched_return',
            'roll_consistent_return', 'removed_level_jump_pp'
        ]]
    )

# Managed-money net position and normalizations.
wide['MM_net'] = wide['MM_L'] - wide['MM_S']

if 'TO_T' in wide.columns:
    wide['total_oi'] = wide['TO_T']
elif 'TO_L' in wide.columns:
    wide['total_oi'] = wide['TO_L']
else:
    wide['total_oi'] = wide['CM_L'] + wide['NC_L'] + wide['NC_X'] + wide['NR_L']

wide['MM_net_frac'] = wide['MM_net'] / wide['total_oi'].where(wide['total_oi'].ne(0))

# Full trailing ~3-year weekly range. No shortened warm-up is labeled "3-year".
window_3y = 156
roll = wide['MM_net'].rolling(window_3y, min_periods=window_3y)
wide['MM_net_min3y'] = roll.min()
wide['MM_net_max3y'] = roll.max()
den = wide['MM_net_max3y'] - wide['MM_net_min3y']
wide['COT_index'] = (wide['MM_net'] - wide['MM_net_min3y']) / den.where(den.ne(0))

# Audit exactly why the 3-year COT-index sample is smaller than the other
# signals. We intentionally require 156 consecutive report rows with valid
# MM_net and never interpolate CFTC positions; one missing report can therefore
# invalidate up to 156 trailing windows.
row_number = pd.Series(np.arange(1, len(wide) + 1), index=wide.index)
full_history = row_number.ge(window_3y)
rolling_valid_mm = wide['MM_net'].rolling(window_3y, min_periods=1).count()
incomplete_3y_window = full_history & rolling_valid_mm.lt(window_3y)
zero_3y_range = full_history & rolling_valid_mm.eq(window_3y) & den.eq(0)
invalid_after_warmup = full_history & wide['COT_index'].isna()

assert (invalid_after_warmup == (incomplete_3y_window | zero_3y_range)).all()

mm_missing_dates = pd.DataFrame({
    'date': wide.index[wide['MM_net'].isna()],
    'MM_L_missing': wide.loc[wide['MM_net'].isna(), 'MM_L'].isna().to_numpy(),
    'MM_S_missing': wide.loc[wide['MM_net'].isna(), 'MM_S'].isna().to_numpy(),
})

cot_index_audit = pd.DataFrame({
    'metric': [
        'COT rows',

```

```

'raw MM net missing rows',
'mechanical 155-row warm-up',
'post-warm-up rows invalid from incomplete 156-row window',
'post-warm-up rows invalid from zero 3-year range',
'valid COT-index rows',
],
'count': [
    len(wide),
    int(wide['MM_net'].isna().sum()),
    min(window_3y - 1, len(wide)),
    int(incomplete_3y_window.sum()),
    int(zero_3y_range.sum()),
    int(wide['COT_index'].notna().sum()),
],
})
print('COT-index attrition audit (strict 156 consecutive reports; no interpolation):')
display(cot_index_audit)
print('Raw MM_net missing report dates:')
if mm_missing_dates.empty:
    print(' none')
else:
    display(mm_missing_dates)

# Raw nearby price level for part (b). Permit only a bounded backward match so
# a short holiday gap can use the prior settlement while long gaps stay missing.
tue_px_match = bounded_asof(
    wide.index, front_px, max_age_days=3, value_name='settlement'
)
wide['price'] = tue_px_match['settlement'].to_numpy()
wide['price_date'] = tue_px_match['obs_date'].to_numpy()
wide['price_age_days'] = tue_px_match['age_days'].to_numpy()

print('First valid 3-year COT-index date:', wide['COT_index'].first_valid_index())
print(f'Price-matched COT weeks: {wide["price"].notna().sum()} of {len(wide)}')
print('Price-match age (days):')
print(wide['price_age_days'].value_counts(dropna=False).sort_index().to_string())

```

Code cell 11

```

# — Plot: MM net vs price —
fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True)

# Top panel: front-month price + managed-money net positioning
ax = axes[0]
ax.plot(wide.index, wide['price'], color='black', lw=1)
ax.set_ylabel('CL front-month ($/bbl)')
ax.set_title('WTI Crude Oil: Front-Month Price and Managed-Money Positioning')
ax.grid(alpha=0.2)

ax2 = ax.twinx()

# Convert pandas nullable values to ordinary NumPy NaNs.
# Missing MM observations remain missing rather than being plotted as zero.
_mm = wide['MM_net'].to_numpy(dtype=float, na_value=np.nan)
_valid = np.isfinite(_mm)

ax2.fill_between(
    wide.index,
    _mm,
    0,
    where=_valid & (_mm >= 0),
    color='green',
    alpha=0.3,
    label='MM net long'
)

ax2.fill_between(
    wide.index,
    _mm,
    0,
    where=_valid & (_mm < 0),
    color='red',
    alpha=0.3,
    label='MM net short'
)

ax2.set_ylabel('MM net position (contracts)')
ax2.legend(loc='upper left', fontsize=7)

# Middle panel: MM net as a fraction of total open interest
ax = axes[1]
ax.plot(wide.index, wide['MM_net_frac'], color='steelblue', lw=1)
ax.axhline(0, color='0.4', lw=0.5)
ax.set_ylabel('MM net / total OI')
ax.set_title('Managed-Money Net as Fraction of Total Open Interest')
ax.grid(alpha=0.2)

# Bottom panel: trailing 3-year COT index
ax = axes[2]
ax.plot(wide.index, wide['COT_index'], color='darkorange', lw=1)
ax.axhline(0.5, color='0.4', lw=0.5, ls='--')
ax.set_ylabel('COT Index [0, 1]')
ax.set_title('COT Index: MM Net in Its Trailing 3-Year Range')
ax.set_ylim(-0.05, 1.05)
ax.grid(alpha=0.2)

fig.tight_layout()

```

```
plt.show()
```

Code cell 12

```
# Hedging pressure: HP = (CM_S - CM_L) / (CM_S + CM_L)
wide['HP'] = (wide['CM_S'] - wide['CM_L']) / (wide['CM_S'] + wide['CM_L'])

frac_net_short = (wide['HP'] > 0).mean()
print(f'Fraction of weeks commercials are net short (HP > 0): {frac_net_short:.1%}')
print(f'Mean HP: {wide["HP"].mean():.4f}')
print(f'Median HP: {wide["HP"].median():.4f}')

fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(wide.index, wide['HP'], color='darkred', lw=1)
ax.axhline(0, color='0.4', lw=0.8)
ax.set_ylabel('Hedging Pressure')
ax.set_title(f'Commercial Hedging Pressure in CL '
             f'(net short {frac_net_short:.0%} of weeks)')
ax.grid(alpha=0.2)
fig.tight_layout()
plt.show()
```

Code cell 13

```
# Tuesday-to-Tuesday return of the continuously rolled nearby strategy.
# The target-date matcher handles short exchange-calendar gaps, while the
# underlying return index removes contract-level jumps at rolls.
tue_ri_match = bounded_asof(
    wide.index, front_return_index, max_age_days=3, value_name='return_index'
)
wide['return_index_tue'] = tue_ri_match['return_index'].to_numpy()
wide['return_index_tue_date'] = tue_ri_match['obs_date'].to_numpy()
wide['return_index_tue_age_days'] = tue_ri_match['age_days'].to_numpy()
wide['ret_tue'] = wide['return_index_tue'].pct_change(fill_method=None)
wide['dMM_net'] = wide['MM_net'].diff()

df_contemp = (wide[['ret_tue', 'dMM_net']]
              .replace([np.inf, -np.inf], np.nan)
              .dropna())

corr = df_contemp['ret_tue'].corr(df_contemp['dMM_net'])
x_pct = df_contemp['ret_tue'] * 100.0 # one unit = one percentage point
y = df_contemp['dMM_net']
slope, intercept, r, p, se = sp_stats.linregress(x_pct, y)

print(f'n = {len(df_contemp)}')
print(f'Contemporaneous correlation: {corr:+.4f}')
print(f'Regression slope: {slope:,.1f} contracts per 1% return')
print(f'OLS t-stat of slope: {slope / se:+.2f}')
print(f'R² = {r**2:.4f}')

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(x_pct, y, alpha=0.3, s=8)
xf = np.linspace(x_pct.quantile(0.01), x_pct.quantile(0.99), 100)
ax.plot(xf, intercept + slope * xf, lw=1.5,
        label=f'slope = {slope:,.0f} contracts per 1%')
ax.set_xlabel('Tuesday-to-Tuesday roll-consistent CL return (%)')
ax.set_ylabel('ΔMM net (contracts)')
ax.set_title(f'Contemporaneous association: ρ = {corr:.3f}')
ax.legend()
ax.grid(alpha=0.2)
fig.tight_layout()
plt.show()
```

Code cell 14

```
def correlation_diagnostics(x, y, hac_lags=4):
    """Correlation plus iid and HAC slope t-stats for a weekly diagnostic."""
    tmp = pd.concat([x.rename('x'), y.rename('y')], axis=1).dropna()
    n = len(tmp)
    if n < 3:
        return np.nan, np.nan, np.nan, n

    rho = tmp['x'].corr(tmp['y'])
    X = sm.add_constant(tmp['x'].to_numpy(dtype=float))
    Y = tmp['y'].to_numpy(dtype=float)
    iid = sm.OLS(Y, X).fit()
    hac = sm.OLS(Y, X).fit(
        cov_type='HAC',
        cov_kws={'maxlags': hac_lags, 'use_correction': True},
    )
    return rho, iid.tvalues[1], hac.tvalues[1], n

checks = [
    ('same week: ret[t] vs dMM[t]', wide['ret_tue'], wide['dMM_net']),
    ('price leads one week: ret[t-1] vs dMM[t]', wide['ret_tue'].shift(1), wide['dMM_net']),
    ('positioning leads one week: dMM[t-1] vs ret[t]', wide['dMM_net'].shift(1), wide['ret_tue']),
]

# Four weekly HAC lags are a one-month robustness diagnostic. These lead/lag
```

```
# regressions are descriptive; no causal direction is inferred from them.
LEADLAG HAC LAGS = 4
leadlag = []
for label, x, y in checks:
    rho_i, iid_t_i, hac_t_i, n_i = correlation_diagnostics(
        x, y, hac_lags=LEADLAG_HAC_LAGS
    )
    leadlag.append({
        'channel': label,
        'rho': rho_i,
        'iid_t_diagnostic': iid_t_i,
        'HAC_t_diagnostic': hac_t_i,
        'HAC_lags': LEADLAG_HAC_LAGS,
        'n': n_i,
    })
leadlag = pd.DataFrame(leadlag)

display(leadlag.style.format({
    'rho': '{:+.4f}',
    'iid_t_diagnostic': '{:+.2f}',
    'HAC_t_diagnostic': '{:+.2f}',
    'HAC_lags': '{:,.0f}',
    'n': '{:,.0f}',
}))

strongest = leadlag.iloc[leadlag['rho'].abs().argmax()]
print(f'Largest absolute correlation: {strongest["channel"]} ({strongest["rho"]:+.4f})')
```

Code cell 15

```
# Assignment convention: returns begin on the Friday after the Tuesday COT
# snapshot. Exact calendar horizons are matched to the latest prior return-index
# observation within three calendar days.
wide['friday'] = pd.DatetimeIndex(wide.index) + pd.offsets.Week(weekday=4)

start_match = bounded_asof(
    wide['friday'], front_return_index, max_age_days=3, value_name='return_index'
)
wide['return_index_fri'] = start_match['return_index'].to_numpy()
wide['return_index_fri_date'] = start_match['obs_date'].to_numpy()
wide['return_index_fri_age_days'] = start_match['age_days'].to_numpy()

for h in [1, 4, 13]:
    target_end = wide['friday'] + pd.to_timedelta(7 * h, unit='D')
    end_match = bounded_asof(
        target_end, front_return_index, max_age_days=3, value_name='return_index'
    )

    wide['fend_target_{h}w'] = target_end
    wide['fend_return_index_{h}w'] = end_match['return_index'].to_numpy()
    wide['fend_return_index_date_{h}w'] = end_match['obs_date'].to_numpy()
    wide['fend_age_days_{h}w'] = end_match['age_days'].to_numpy()

    valid = (wide['return_index_fri'].notna() &
              wide['fend_return_index_{h}w'].notna() &
              (wide['fend_return_index_date_{h}w'] > wide['return_index_fri_date']))

    wide['ffwd_ret_{h}w'] = np.nan
    wide.loc[valid, 'ffwd_ret_{h}w'] = (
        wide.loc[valid, 'fend_return_index_{h}w'] /
        wide.loc[valid, 'return_index_fri'] - 1.0
    )

# Tradable-timing robustness: the COT report is published after the Friday CL
# settlement. With daily data, use the first settlement STRICTLY after that
# Friday as the first certainly observable post-publication price. Then hold
# for the same h calendar weeks from that observed start date.
post_start = bounded_forward_asof(
    wide['friday'], front_return_index, max_delay_days=4,
    value_name='post_release_return_index'
)
wide['post_release_start_index'] = post_start['post_release_return_index'].to_numpy()
wide['post_release_start_date'] = post_start['obs_date'].to_numpy()
wide['post_release_start_delay_days'] = post_start['delay_days'].to_numpy()

for h in [1, 4, 13]:
    post_target_end = pd.to_datetime(wide['post_release_start_date']) + pd.to_timedelta(7 * h, unit='D')
    post_end_match = bounded_asof(
        post_target_end, front_return_index, max_age_days=3,
        value_name='post_release_end_index'
    )
    wide['fpost_end_target_{h}w'] = post_target_end
    wide['fpost_end_index_{h}w'] = post_end_match['post_release_end_index'].to_numpy()
    wide['fpost_end_date_{h}w'] = post_end_match['obs_date'].to_numpy()
    wide['fpost_end_age_days_{h}w'] = post_end_match['age_days'].to_numpy()

    valid = (wide['post_release_start_index'].notna() &
              wide['fpost_end_index_{h}w'].notna() &
              (wide['fpost_end_date_{h}w'] > wide['post_release_start_date']))
    wide['fpost_fwd_ret_{h}w'] = np.nan
    wide.loc[valid, 'fpost_fwd_ret_{h}w'] = (
```

```

wide.loc[valid, f'post_end_index_{h}w'] /
wide.loc[valid, 'post_release_start_index'] - 1.0
)

print('Roll-return index history:', front_return_index.index.min().date(), 'to',
      front_return_index.index.max().date())
print('Last COT snapshot:', wide.index.max().date())
print('Friday assignment-start match ages:')
print(wide['return_index_fri_age_days'].value_counts(dropna=False).sort_index().to_string())
print('Post-release start delays (strictly after Friday):')
print(wide['post_release_start_delay_days'].value_counts(dropna=False).sort_index().to_string())

```

Code cell 16

```

signals = [
    ('MM net / OI', 'MM_net_frac'),
    ('COT index', 'COT_index'),
    ('Hedging pressure', 'HP'),
]

def run_predictive_regressions(return_prefix):
    """Three signals x three horizons with OLS and overlap-aware HAC inference."""
    results = []
    for sig_name, sig_col in signals:
        for h in [1, 4, 13]:
            ret_col = return_prefix.format(h=h)
            tmp = (wide[[sig_col, ret_col]]
                  .replace([np.inf, -np.inf], np.nan)
                  .dropna()
                  .copy())
            if len(tmp) < 30:
                continue

            sig_sd = tmp[sig_col].std(ddof=1)
            if not np.isfinite(sig_sd) or sig_sd == 0:
                continue

            x = (tmp[sig_col] - tmp[sig_col].mean()) / sig_sd
            y = tmp[ret_col]
            X = sm.add_constant(x.to_numpy(dtype=float))

            ols = sm.OLS(y.to_numpy(dtype=float), X).fit()
            hac_lags = max(h - 1, 0)
            hac = sm.OLS(y.to_numpy(dtype=float), X).fit(
                cov_type='HAC',
                cov_kwds={'maxlags': hac_lags, 'use_correction': True},
            )

            results.append({
                'signal': sig_name,
                'horizon_weeks': h,
                'n': len(tmp),
                'IC': x.corr(y),
                'slope_pct_per_1sd': 100.0 * ols.params[1],
                'OLS_t': ols.tvalues[1],
                'HAC_t': hac.tvalues[1],
                'HAC_lags': hac_lags,
            })
    return (pd.DataFrame(results)
            .sort_values(['signal', 'horizon_weeks'])
            .reset_index(drop=True))

fmt = {
    'IC': '{:+.4f}'.format,
    'slope_pct_per_1sd': '{:+.3f}'.format,
    'OLS_t': '{:+.2f}'.format,
    'HAC_t': '{:+.2f}'.format,
}

# Primary table: exact Friday convention requested by the assignment.
res_df = run_predictive_regressions(fwd_ret_{h}w')
print('Assignment convention: Friday-to-horizon returns')
display(res_df.assign(**{c: res_df[c].map(f) for c, f in fmt.items()}))

# Robustness table: first daily settlement strictly after Friday publication.
res_post_df = run_predictive_regressions(post_fwd_ret_{h}w')
print('Post-release robustness: first settlement strictly after release Friday')
display(res_post_df.assign(**{c: res_post_df[c].map(f) for c, f in fmt.items()}))

# Side-by-side timing sensitivity for the two inference conventions.
timing_compare = res_df.merge(
    res_post_df,
    on=['signal', 'horizon_weeks'],
    suffixes=('_assignment', '_postrelease'),
    how='outer',
)
cols = [

```

```

'signal', 'horizon_weeks',
'n_assignment', 'IC_assignment', 'HAC_t_assignment',
'n_postrelease', 'IC_postrelease', 'HAC_t_postrelease',
]
print('Timing-robustness comparison')
display(timing_compare[cols].style.format({
    'IC_assignment': '{:+.4f}',
    'HAC_t_assignment': '{:+.2f}',
    'IC_postrelease': '{:+.4f}',
    'HAC_t_postrelease': '{:+.2f}',
    'n_assignment': '{:,.0f}',
    'n_postrelease': '{:,.0f}',
}))

```

Code cell 17

```

# Alignment / stale-data audit. Fail explicitly if timing invariants are violated.
audit_rows = []
post_audit_rows = []
for h in [1, 4, 13]:
    valid = wide[f'fwd_ret_{h}w'].notna()
    if valid.any():
        starts = wide.loc[valid, 'friday']
        start_obs_dates = pd.to_datetime(wide.loc[valid, 'return_index_fri_date'])
        ends = pd.to_datetime(wide.loc[valid, f'end_target_{h}w'])
        end_obs_dates = pd.to_datetime(wide.loc[valid, f'end_return_index_date_{h}w'])

        assert (start_obs_dates <= starts).all()
        assert (end_obs_dates <= ends).all()
        assert ((starts - start_obs_dates).dt.days <= 3).all()
        assert ((ends - end_obs_dates).dt.days <= 3).all()
        assert (end_obs_dates > start_obs_dates).all()
        assert (end_obs_dates <= front_return_index.index.max()).all()

    audit_rows.append({
        'horizon_weeks': h,
        'n_returns': int(valid.sum()),
        'last_start_target': starts.max().date(),
        'last_end_observation': end_obs_dates.max().date(),
        'max_start_age_days': int((starts - start_obs_dates).dt.days.max()),
        'max_end_age_days': int((ends - end_obs_dates).dt.days.max()),
    })

    pvalid = wide[f'post_fwd_ret_{h}w'].notna()
    if pvalid.any():
        release_fridays = pd.to_datetime(wide.loc[pvalid, 'friday'])
        post_starts = pd.to_datetime(wide.loc[pvalid, 'post_release_start_date'])
        post_targets = pd.to_datetime(wide.loc[pvalid, f'post_end_target_{h}w'])
        post_ends = pd.to_datetime(wide.loc[pvalid, f'post_end_date_{h}w'])

        assert (post_starts > release_fridays).all()
        assert ((post_starts - release_fridays).dt.days.between(1, 4)).all()
        assert (post_ends <= post_targets).all()
        assert ((post_targets - post_ends).dt.days.between(0, 3)).all()
        assert (post_ends > post_starts).all()
        assert (post_ends <= front_return_index.index.max()).all()

    post_audit_rows.append({
        'horizon_weeks': h,
        'n_returns': int(pvalid.sum()),
        'max_release_to_start_days': int((post_starts - release_fridays).dt.days.max()),
        'max_end_age_days': int((post_targets - post_ends).dt.days.max()),
        'last_start_observation': post_starts.max().date(),
        'last_end_observation': post_ends.max().date(),
    })

print('Assignment-timing alignment audit')
audit_df = pd.DataFrame(audit_rows)
display(audit_df)
print('Post-release timing alignment audit')
post_release_audit_df = pd.DataFrame(post_audit_rows)
display(post_release_audit_df)

```

Code cell 18

```

fig, axes = plt.subplots(3, 3, figsize=(14, 11))

for j, (sig_name, sig_col) in enumerate(signals):
    for k, h in enumerate([1, 4, 13]):
        ax = axes[j, k]
        ret_col = f'fwd_ret_{h}w'
        tmp = wide[[sig_col, ret_col]].replace([np.inf, -np.inf, np.nan]).dropna().copy()

        if len(tmp) < 30 or tmp[sig_col].std(ddof=1) == 0:
            ax.set_axis_off()
            continue

        x = (tmp[sig_col] - tmp[sig_col].mean()) / tmp[sig_col].std(ddof=1)
        y_pct = tmp[ret_col] * 100.0
        slope_i, intercept_i, _, _ = sp_stats.linregress(x, y_pct)

```

```

ax.scatter(x, y_pct, alpha=0.2, s=5)
xf = np.linspace(x.quantile(0.01), x.quantile(0.99), 50)
ax.plot(xf, intercept_i + slope_i * xf, lw=1.2)

row = res_df[(res_df['signal'].eq(sig_name)) &
              (res_df['horizon_weeks'].eq(h))]
if not row.empty:
    row = row.iloc[0]
    ax.set_title(f'{sig_name} → {h}-week return\n'
                 f'IC={row.IC:+.3f}, OLS t={row.OLS_t:+.2f}, HAC t={row.HAC_t:+.2f}',
                 fontsize=8)
    ax.set_xlabel('standardized signal', fontsize=7)
    ax.set_ylabel('forward return (%)', fontsize=7)
    ax.grid(alpha=0.2)

fig.suptitle('Predictive Regressions: COT Signals → Forward CL Returns', fontsize=11)
fig.tight_layout()
plt.show()

```

Code cell 19

```

# Final Q1 checks.
assert not front.index.duplicated().any()
assert wide['COT_index'].dropna().between(0, 1).all()
assert wide['price_age_days'].dropna().between(0, 3).all()
assert wide['return_index_tue_age_days'].dropna().between(0, 3).all()
assert wide['return_index_fri_age_days'].dropna().between(0, 3).all()
assert front_return_index.index.is_monotonic_increasing
assert not front_return_index.index.duplicated().any()
assert front_return_index.notna().all()
assert front_daily_ret.iloc[1:].notna().all()

if not roll_return_audit.empty:
    assert roll_return_audit['anchor_age_days'].between(0, 3).all()
    assert (roll_return_audit['anchor_date'] <=
            roll_return_audit['scheduled_roll_date']).all()

for h in [1, 4, 13]:
    assert wide[f'end_age_days_{h}w'].dropna().between(0, 3).all()
    valid = wide[f'fwd_ret_{h}w'].notna()
    if valid.any():
        assert (pd.to_datetime(wide.loc[valid, f'end_return_index_date_{h}w']) >
                pd.to_datetime(wide.loc[valid, f'return_index_fri_date'])).all()

assert wide['post_release_start_delay_days'].dropna().between(1, 4).all()
for h in [1, 4, 13]:
    assert wide[f'post_end_age_days_{h}w'].dropna().between(0, 3).all()
    valid = wide[f'post_fwd_ret_{h}w'].notna()
    if valid.any():
        assert (pd.to_datetime(wide.loc[valid, 'post_release_start_date']) >
                pd.to_datetime(wide.loc[valid, 'friday'])).all()
        assert (pd.to_datetime(wide.loc[valid, f'post_end_date_{h}w']) >
                pd.to_datetime(wide.loc[valid, 'post_release_start_date'])).all()

print('Snapshot caps:')
print(f' COT <= {COT_AS_OF_DATE:%Y-%m-%d}; SHA256={COT_SHA256}')
print(f' CL settlements <= {CL_AS_OF_DATE:%Y-%m-%d}; SHA256={CL_PRICE_SHA256}')
print('Q1 data, roll, attrition, and timing checks passed.')

```

Code cell 20

```

conn.close()
print('WRDS connection closed.')

```

Code cell 21

```

# Question 2 uses the instructor-supplied futsread package AS PROVIDED.
# Do not patch or replace the package to run this notebook.
PROJECT_DIR = Path.cwd()
if str(PROJECT_DIR) not in sys.path:
    sys.path.insert(0, str(PROJECT_DIR))

import futsread
from futsread import (
    build_combo, load_combo, condense, summary, stats,
    plot_combo, parse_combo, download,
)

print(f'Instructor futsread loaded from: {Path(futsread.__file__).resolve()}')
print(f'download signature: {inspect.signature(download)}')
print('Instructor package is used unchanged; notebook-level audits below enforce '
      'the assignment-specific full-window requirement.')

for spread_code in ['NG[H][J0]', 'ZC[U][Z0]', 'ZW[K][N0]', 'HE[V][Z0]']:
    print(parse_combo(spread_code))

```

```

def print_spread_diagnostics(cond, per, st, expected_direction):
    # Print cross-year statistics, hypothesis agreement, exceptions, and timing.
    if expected_direction not in {'down', 'up'}:
        raise ValueError("expected direction must be 'down' or 'up'")

    pnl = per['pnl'].dropna()
    if expected_direction == 'down':
        agrees = pnl < 0
        exceptions = per.loc[per['pnl'] > 0, ['instance', 'pnl']]
        agreement_label = 'negative full-window P&L'
        exception_label = 'Positive (opposite-direction) years'
    else:
        agrees = pnl > 0
        exceptions = per.loc[per['pnl'] < 0, ['instance', 'pnl']]
        agreement_label = 'positive full-window P&L'
        exception_label = 'Negative (opposite-direction) years'

    flat = pnl.eq(0)
    n = len(pnl)
    n_agree = int(agrees.sum())
    agreement_rate = n_agree / n if n else np.nan

    print(f'n={st['n']}, mean={st['mean']:.4f}, median={st['median']:.4f}, '
          f'positive_rate={st['hit_rate']:.1%}, t={st['t_stat']:.2f}')
    print(f'Scenario agreement ({agreement_label}): '
          f'{n_agree}/{n} = {agreement_rate:.1%}')
    if flat.any():
        print(f'Exactly flat full-window years: {int(flat.sum())}')

    print(f'\n{exception_label}:')
    if exceptions.empty:
        print(' none')
    else:
        print(exceptions.to_string(index=False))

    landmarks = [x for x in (-120, -60, -40, -20, -10, 0) if x in cond.index]
    timing = cond.loc[landmarks, ['mean', 'median', 'n']].copy()
    print("\nCondensed path at selected trading-day offsets:")
    print(timing.to_string(float_format=lambda x: f'{x:.4f}'))

def assert_strict_full_window(wide_matrix, window):
    # Notebook-level audit: every retained annual instance must cover every offset.
    lo, hi = window
    assert not wide_matrix.empty
    assert wide_matrix.index.min() == lo
    assert wide_matrix.index.max() == hi
    assert list(wide_matrix.index) == list(range(lo, hi + 1))
    assert wide_matrix.notna().all().all()
    print(f'Strict full-window check passed: {wide_matrix.shape[1]} instances x '
          f'{wide_matrix.shape[0]} offsets [{lo}, {hi}]')

def print_expiry_source_audit(df):
    # Surface the instructor package's terminal-date conventions; do not alter them.
    cols = [c for c in ['instance', 'ltd', 'ltd_is_fnd', 'fnd_is_rule', 'ltd_source']
            if c in df.columns]
    audit = df[cols].drop_duplicates('instance').sort_values('instance')
    print('Instructor-package terminal-date source audit:')
    group_cols = [c for c in ['ltd_source', 'fnd_is_rule'] if c in audit.columns]
    if group_cols:
        display(audit.groupby(group_cols, dropna=False)
                 .size().rename('n_instances').reset_index())
    else:
        display(audit.head())
    return audit

```

Code cell 22

```

# Run once if the four required series are not already in the local futsread store.
# This is the instructor package's original API; no custom as_of date argument is used.
spread_info = download(WRDS_USER, tickers=['NG', 'ZC', 'ZW', 'HE'])

print('Downloaded/available spread metadata rows:', len(spread_info))
if 'fnd_is_rule' in spread_info.columns:
    print('Instructor-package rule-derived FND count:',
          int(spread_info['fnd_is_rule'].fillna(False).sum()))

```

Code cell 23

```

combos = ['NG[H][J0]', 'ZC[U][Z0]', 'ZW[K][N0]', 'HE[V][Z0]']
for spread_code in combos:
    df = build_combo(spread_code)
    n = df['instance'].nunique() if not df.empty else 0
    print(f'{spread_code}: {n} instances, {len(df):,} rows')

```

Code cell 24

```

df_ng = load_combo('NG[H][J0]')

```

```

window_ng = (-120, 0)

fig, (cond_ng, wide_ng, per_ng, st_ng) = plot_combo(
    df_ng, window=window_ng,
    require_full=True, min_coverage=1.0,
    title='NG[H][J0] Mar/Apr Natural Gas (Widow-Maker)',
    outfile='ng_widow_maker.png',
)
plt.show()

assert_strict_full_window(wide_ng, window_ng)
expiry_audit_ng = print_expiry_source_audit(df_ng)
print_spread_diagnostics(cond_ng, per_ng, st_ng, expected_direction='down')

```

Code cell 25

```

df_zc = load_combo('ZC[U][Z0]')
window_zc = (-120, 0)

fig, (cond_zc, wide_zc, per_zc, st_zc) = plot_combo(
    df_zc, window=window_zc,
    require_full=True, min_coverage=1.0,
    title='ZC[U][Z0] Sep/Dec Corn (Old-Crop / New-Crop)',
    outfile='zc_harvest.png',
)
plt.show()

assert_strict_full_window(wide_zc, window_zc)
expiry_audit_zc = print_expiry_source_audit(df_zc)
print_spread_diagnostics(cond_zc, per_zc, st_zc, expected_direction='down')

```

Code cell 26

```

df_zw = load_combo('ZW[K][N0]')
window_zw = (-120, 0)

fig, (cond_zw, wide_zw, per_zw, st_zw) = plot_combo(
    df_zw, window=window_zw,
    require_full=True, min_coverage=1.0,
    title='ZW[K][N0] May/Jul SRW Wheat (Old-Crop / New-Crop)',
    outfile='zw_harvest.png',
)
plt.show()

assert_strict_full_window(wide_zw, window_zw)
expiry_audit_zw = print_expiry_source_audit(df_zw)
print_spread_diagnostics(cond_zw, per_zw, st_zw, expected_direction='down')

```

Code cell 27

```

df_he = load_combo('HE[V][Z0]')
window_he = (-120, 0)

fig, (cond_he, wide_he, per_he, st_he) = plot_combo(
    df_he, window=window_he,
    require_full=True, min_coverage=1.0,
    title='HE[V][Z0] Oct/Dec Lean Hogs (Cash Settlement Convergence)',
    outfile='he_convergence.png',
)
plt.show()

assert_strict_full_window(wide_he, window_he)
expiry_audit_he = print_expiry_source_audit(df_he)
print_spread_diagnostics(cond_he, per_he, st_he, expected_direction='up')

```

Code cell 28

```

# Final reproducibility manifest.
# Question 1 direct WRDS pulls are frozen to the submitted dates above.
# Question 2 uses the instructor-supplied futsread package unchanged; for those
# derived combo files we record the actual date span produced by the run rather
# than claiming an unsupported package-level as-of cap.
repro_manifest = pd.DataFrame([
    {'artifact': 'COT raw query', 'snapshot_or_span': f'<= {COT_AS_OF_DATE:%Y-%m-%d}',
      'rows': len(cot_raw), 'first': cot_raw['date'].min().date(),
      'last': cot_raw['date'].max().date(), 'sha256': COT_SHA256},
    {'artifact': 'CL settlements (Q1)', 'snapshot_or_span': f'<= {CL_AS_OF_DATE:%Y-%m-%d}',
      'rows': len(cl_px), 'first': cl_px['date'].min().date(),
      'last': cl_px['date'].max().date(), 'sha256': CL_PRICE_SHA256},
    {'artifact': 'NG[H][J0] combo rows', 'snapshot_or_span': 'instructor futsread output',
      'rows': len(df_ng), 'first': df_ng['date'].min().date(),
      'last': df_ng['date'].max().date(), 'sha256': None},
    {'artifact': 'ZC[U][Z0] combo rows', 'snapshot_or_span': 'instructor futsread output',
      'rows': len(df_zc), 'first': df_zc['date'].min().date(),
      'last': df_zc['date'].max().date(), 'sha256': None},
    {'artifact': 'ZW[K][N0] combo rows', 'snapshot_or_span': 'instructor futsread output',
      'rows': len(df_zw), 'first': df_zw['date'].min().date(),
      'last': df_zw['date'].max().date(), 'sha256': None},
    {'artifact': 'HE[V][Z0] combo rows', 'snapshot_or_span': 'instructor futsread output',
      'rows': len(df_he), 'first': df_he['date'].min().date(),

```

```
'last': df_he['date'].max().date(), 'sha256': None},  
])  
display(repro_manifest)  
print('Python:', platform.python_version())  
print('All assignment assertions completed.')
```